



tenda路由器的漏洞发现 _

📅 April 22, 2022 am

📶 2.4k words 🕒 20 mins

分析这款是tendaA1206，固件是比较早的未加密的那个。

都是些个人学习过程中的思考与知识，整理下来。

固件在这：<https://p1yang.github.io/2022/04/22/iot/tenda路由器的漏洞发现/>

前期准备

都是些老生常谈的东西可以跳过。

这里使用的qemu-user，方便

至于分析的文件在下面思路中会聊到，这里环境模拟启动的是/bin/httpd 文件

复现环境是qemu+ghidra(反编译伪代码，我个人比较习惯ghidra的伪代码)+ida7.5(动态调试)

binwalk解包，文件格式， qemu-user模式启动等这些就不赘述，主要说几个环境模拟时的几个小问题。



问题1

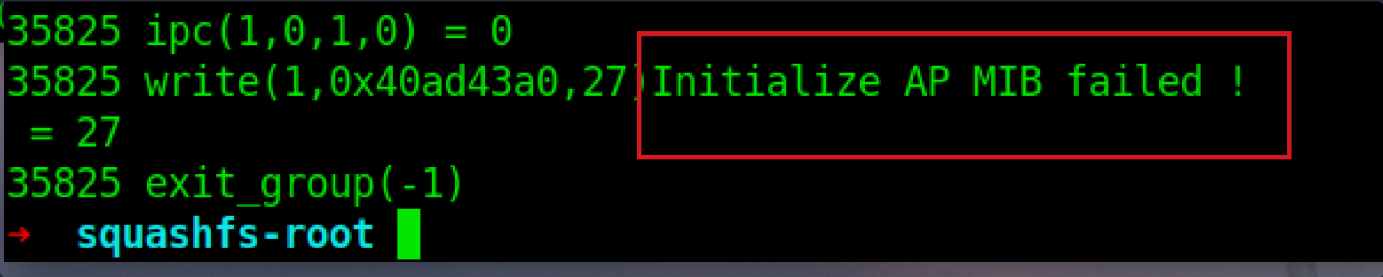


image-20220422093901642

第一次运行时爆出这个错误停止。string大法发现在main中

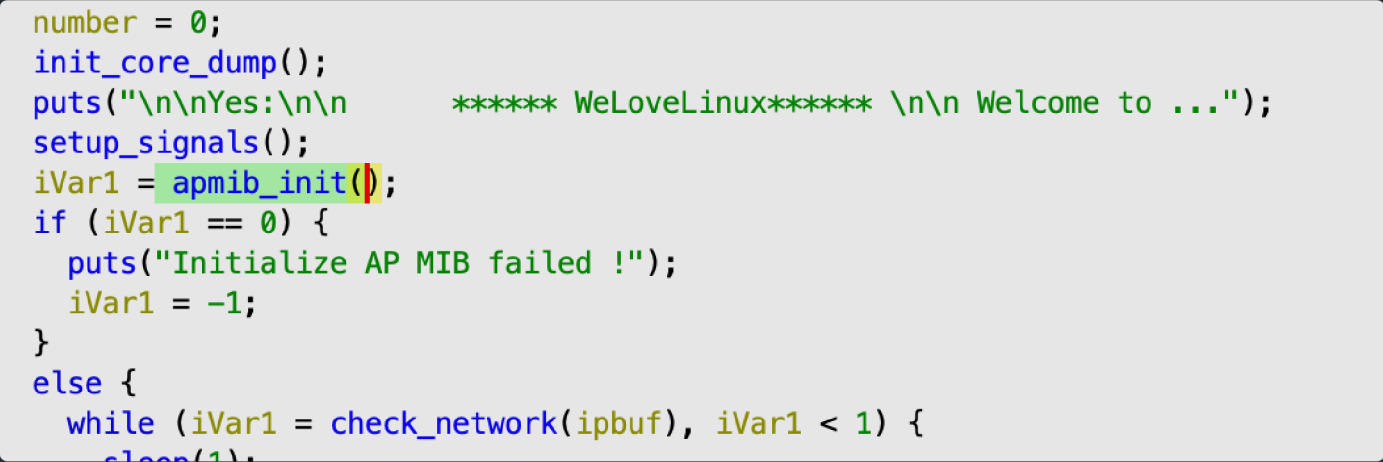


image-20220422094037256

apmib_init 函数从flash中读取mib值到RAM中，像这种模拟是办不到的东西，直接patch代码或更改寄存器值来绕过(尝试了下没办法直接patch代码，可以试试patch机器码，比较麻烦，我就直接改寄存器了)

在mips的判断是bne，btgz等，将断点下在他们上，他们通常依靠v0寄存器的值来做判断。



image-20220422094858896

此时v0值为0，改为1跳过

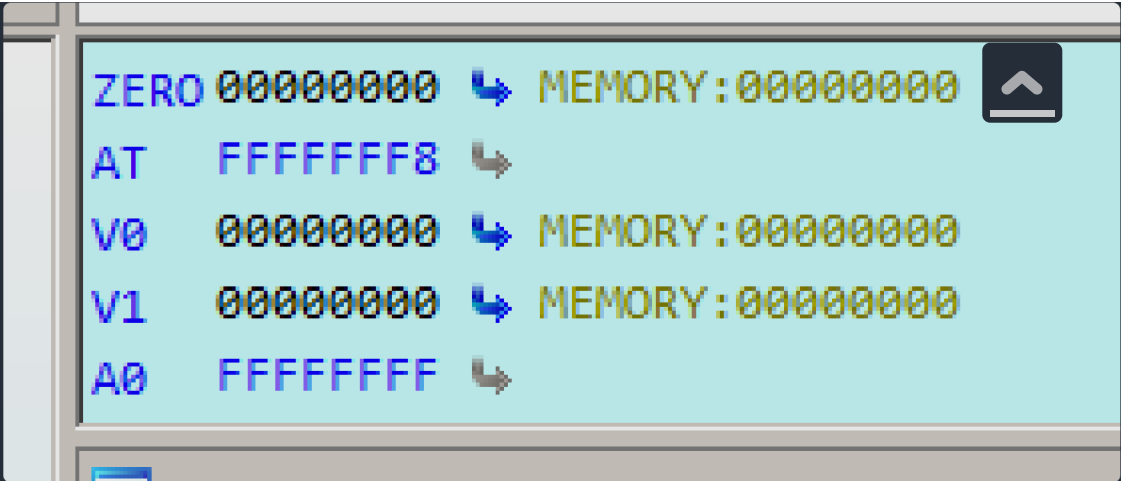


image-20220422095038420

问题2

这里陷入个死循环，问题点在

```
iVar1 = apmib_init();
if (iVar1 == 0) {
    puts("Initialize AP MIB failed !");
    iVar1 = -1;
}
else {
    while (iVar1 = check_network(ipbuf), iVar1 < 1) {
        sleep(1);
    }
    sleep(1);
    iVar1 = ConnectCfm();
    if (iVar1 == 0) {
        printf("connect cfm failed!");
        iVar1 = 0;
    }
    else {
```

image-20220422101918935

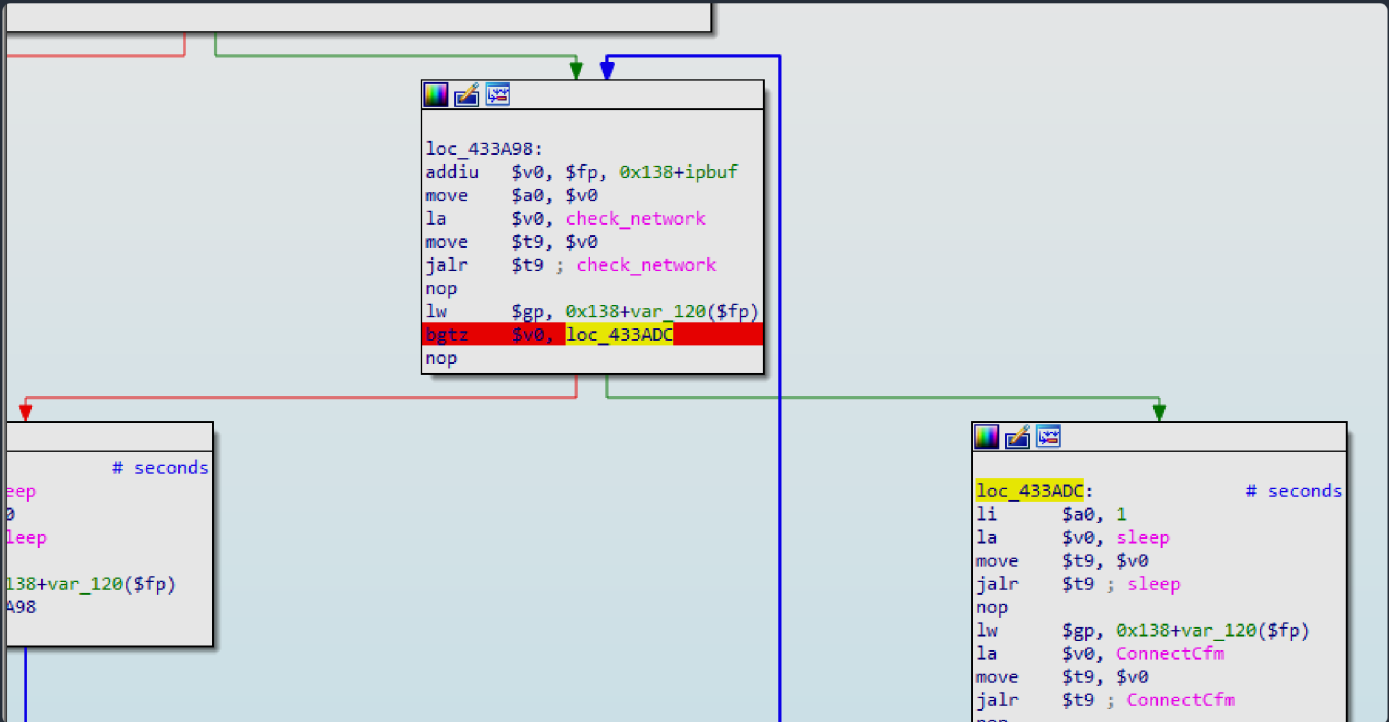


image-20220422102003495

也尝试更改寄存器v0的值成功绕过。

问题3

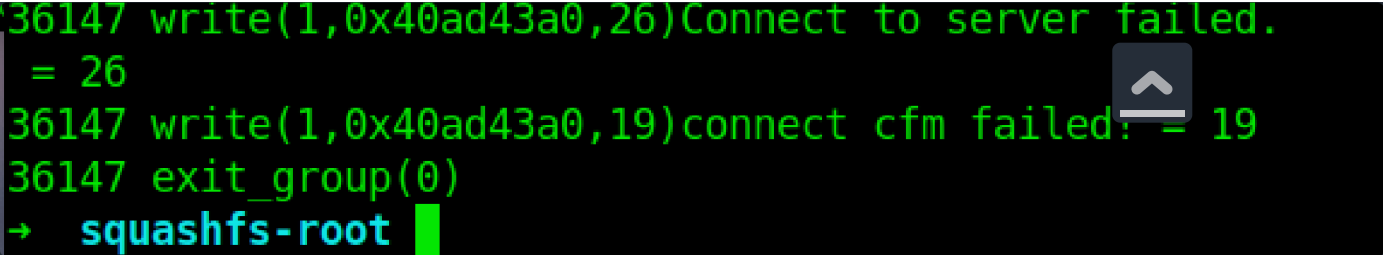


image-20220422095242595

继续string大法



image-20220422095310053

抱歉这里我并没有查到这个函数的是干什么的，有清楚的请告诉我，提前感谢。

不影响，改寄存器大法。

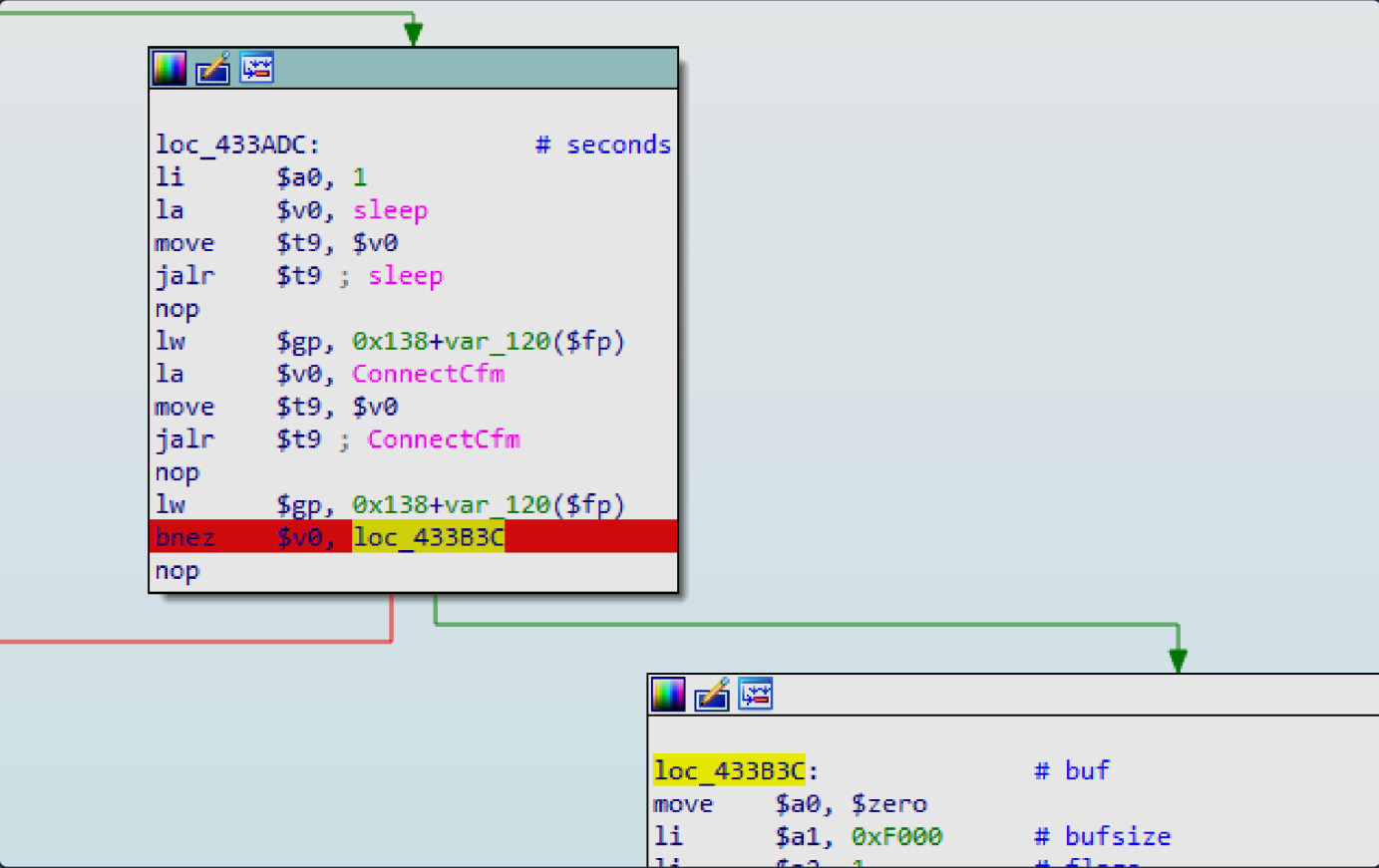


image-20220422095958809

问题4

上面没问题之后发现ip开在 255.255.255.255 上。


```
= -1 errno=99 (Protocol not available)
3527 setsockopt(5,65535,4,1082131196,4,0) = 0
3527 bind(5,1082131180,16,0,0,0) = 0
3527 write(1,0x40ad43a0,44)httpd listen ip = 255.255.255.255 port = 80
= 44
3527 listen(5,128,0,0,0,0) = 0
3527 fcntl(5,F_GETFL) = 2
3527 fcntl(5,F_SETFL,O_RDWR|O_NONBLOCK) = 0
```

image-20220422102645100

string大法搜 listen ip

```
else {
    pcVar6 = inet_ntoa((in_addr)sockaddr.sin_addr);
    uVar1 = ntohs(sockaddr.sin_port);
    printf("httpd listen ip = %s port = %d\n",pcVar6,CONCAT22(extraout_var,uVar1));
    if (uVar4 == 0) {
        iVar5 = listen(psVar2->sock,0x80);
        if (iVar5 < 0) {
            socketFree(sid_00);
            return -1;
        }
        psVar2->flags = psVar2->flags | 0x100;
    }
    psVar2->handlerMask = psVar2->handlerMask | 2;
```

image-20220422103034994

inet_ntoa 函数的意思是， 功能是将网络地址转换成"."点隔的字符串格式。

所以跟sockaddr.sin_port有关， 查看引用

```
psVar2 = socketListen(sid_00);
memset(&sockaddr,0,0x10);
sockaddr.sin_family = 2;
sockaddr.sin_port = htons((uint16_t)port);
if (host == (char *)0x0) {
    sockaddr.sin_addr = 0;
}
else {
    sockaddr.sin_addr = inet_addr(host);
}
uVar3 = psVar2->flags & 4;
if (uVar3 != 0) {
    psVar2->flags = psVar2->flags | 0x20;
```

image-20220422103359077

inte_addr 功能是将一个点分十进制的IP转换成一个长整型数（u_long类型） 等同于inet_addr()。

与host有关， 再向前查看

```
int websOpenListen(int port,int retries)
{
    char *pHost;
    int orig;
    int i;
    char br0IP [16];

    br0IP._0_4_ = 0;
    br0IP._4_4_ = 0;
    br0IP._8_4_ = 0;
    br0IP._12_4_ = 0;
    memset(br0IP,0,0x10);
    i = 0;
    while ((i <= retries &&
            (websListenSock = socketOpenConnection(g_lan_ip,port,websAccept,0), websListenSock < 0))) {
        i = i + 1;
    }
    if (retries < i) {
        printf("%s %d: Couldn't open a socket on ports %d\n","websOpenListen",0xfd,port);
        port = -1;
    }
    else {
        websPort = port;
        bfreeSafe(websHostUrl);
    }
}
```

image-20220422103859404

其参数为全局变量 g_lan_ip。设置个lanip

```
sudo tuncctl -t br0 -u ‘用户名’

sudo ifconfig br0 192.168.5.1/24
```

ps eth1就是第二块网卡第一块通常是eth0 tap是虚拟网络接口 br是网桥

这个设置完之后问题2直接解决了。

```
= -1 errno=99 (Protocol not available)
2571 setsockopt(5,65535,4,1082131196,4,0) = 0
2571 bind(5,1082131180,16,0,0,0) = 0
2571 write(1,0x40ad43a0,40)httpd listen ip = 192.168.5.1 port = 80
= 40
```

image-20220422110341615

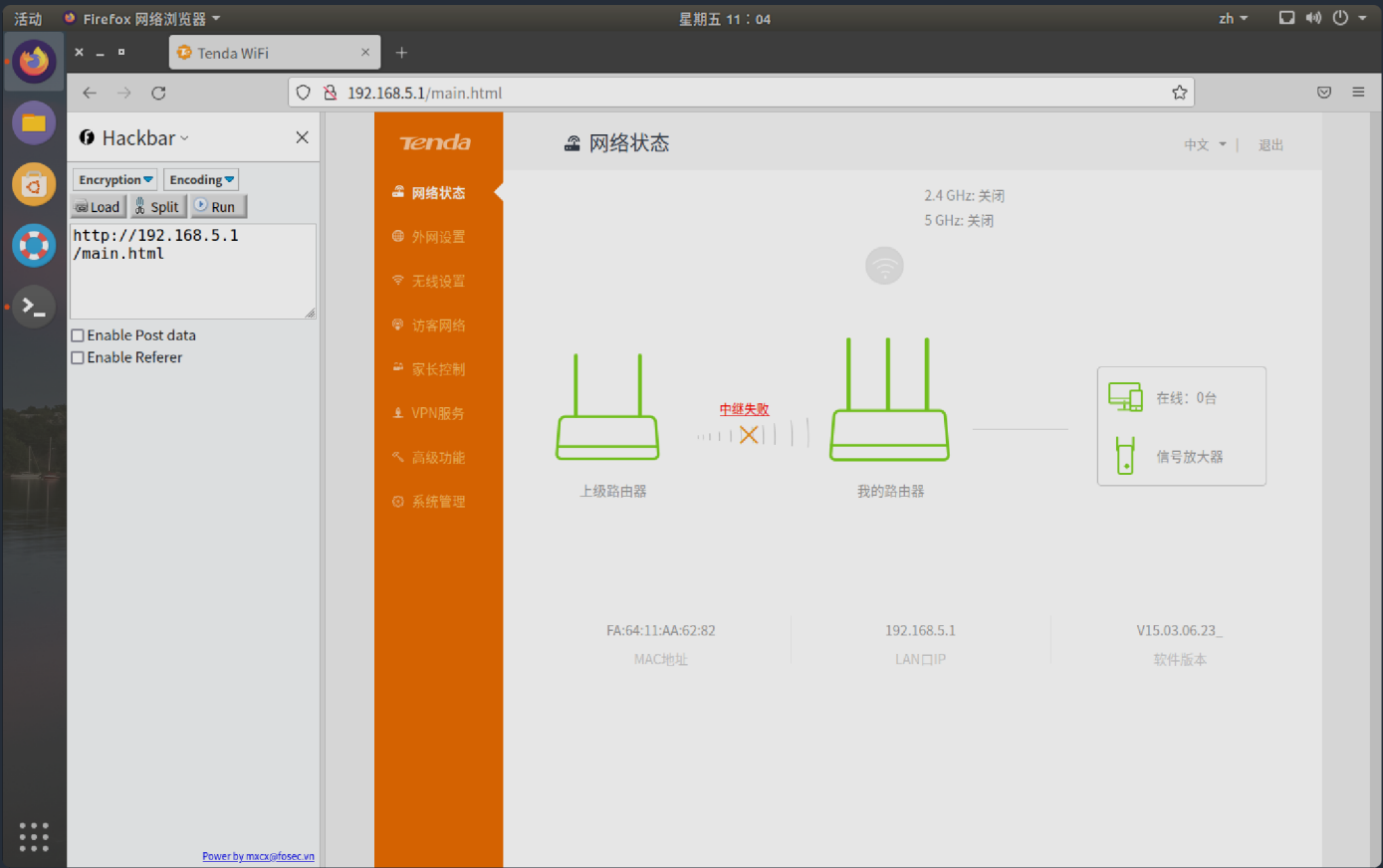


image-20220422110418896

分析思路

分析其使用的web服务器，常见的嵌入式有以下几种：httpd，uhttpd，gohead，lighttpd，boa

还有其他的，我没咋见过，就不写了，用到的话自行查阅（懒！）

我分析这款使用的是httpd，在bin目录下，一般服务器文件都在一下几个目录，不排除其他目录

AWK

```
/user/bin/

/user/sbin/

/bin/
```

在逆向分析httpd时，尽量关注一些自定义功能代码，main下调用的initwebs函数中，配置了前端访问方式

Decompile: main - (httpd)

```
73     uVar2 = getLanIfName();
74     iVar1 = getIfIp(uVar2,br0IP);
75     if (iVar1 < 0) {
76         GetValue("lan.ip",value);
77         strcpy(g_lan_ip,value);
78         memset(&lan_ip_info,0,0x50);
79         iVar1 = tpi_lan_dhcpc_get_ipinfo_and_status(&lan_ip_info);
80         if ((iVar1 == 0) && (lan_ip_info.lan_ip[0] != '\0')) {
81             vos_strcpy(g_lan_ip,&lan_ip_info);
82         }
83     }
84     else {
85         vos_strcpy(g_lan_ip,br0IP);
86     }
87     memset(&info,0,9);
88     info.ip = inet_addr(g_lan_ip);
89     tpi_talk_to_kernel(5,&info,&number,0,0,0);
90     getwebuserpwd(1);
91     getwebuserpwd(0);
92     _Var3 = getpid();
93     doSystemCmd("echo %d > %s",_Var3,"/etc/httpd.pid");
94     iVar1 = initWebs();
95     if (iVar1 < 0) {
96         puts("main -> initWebs failed");
97         iVar1 = -1;
98     }
99     else {
100         memset(loginUserInfo,0,0x6c);
101         signal(0xf,webTaskMainHandler);
```

image-20220422112301584

可以看到默认页面main,

Decompile: initWebs - (httpd)

```
32     if (sVar1 + 1 < 0x80) {
33         sVar1 = strlen(host);
34         iVar2 = sVar1 + 1;
35     }
36     else {
37         iVar2 = 0x80;
38     }
39     ascToUni(wbuf,host,iVar2);
40     websSetHost(wbuf);
41     websSetDefaultPage("main.html");
42     websSetPassword(password);
43     iVar2 = websOpenServer(port,retries);
44     if (iVar2 < 0) {
45         printf("%s %d: websOpenServer failed\n","initWebs",0x208);
46         iVar2 = -1;
47     }
48     else {
49         websUrlHandlerDefine("",(char_t *)0x0,0,R7WebsSecurityHandler,1);
50         websUrlHandlerDefine("/goform",(char_t *)0x0,0,websFormHandler,0);
51         websUrlHandlerDefine("/cgi-bin",(char_t *)0x0,0,webs_Tenda_CGI_BIN_Handler,0);
52         websUrlHandlerDefine("",(char_t *)0x0,0,websDefaultHandler,2);
53         formDefineTendDa();
54         websUrlHandlerDefine("/",(char_t *)0x0,0,websHomePageHandler,0);
55         iVar2 = 0;
56     }
57     return iVar2;
58 }
```

image-20220422112355613

`websSetPassword` 设置访问口令，不多说各位调试的时候可以关注一下。

`websUrlHandlerDefine` 需要关注，这个函数的意思是什么样的url交给谁处理。



上面说了 `尽量关注一些自定义功能代码`

这里的自定义功能代码就在 `formDefineTendDa` 中

```
void formDefineTendDa(void)
{
    websAspDefine("TendaGetLongString", aspTendaGetLongString);
    websAspDefine("aspTendaGetStatus", aspTendaGetStatus);
    websFormDefine("updateUrlLog", updateUrlLog);
    websFormDefine("SysStatusHandle", fromSysStatusHandle);
    websFormDefine("GetWanStatus", formGetWanStatus);
    websFormDefine("GetSysInfo", formGetSysInfo);
    websFormDefine("GetWanStatistic", formGetWanStatistic);
    websFormDefine("GetAllWanInfo", formGetAllWanInfo);
    websFormDefine("GetWanNum", formGetWanNum);
    websAspDefine("aspGetWanNum", aspGetWanNum);
    websFormDefine("getPortStatus", formGetPortStatus);
    websFormDefine("GetSystemStatus", formGetSystemStatus);
    websFormDefine("GetRouterStatus", formGetRouterStatus);
    websFormDefine("setNotUpgrade", formsetNotUpgrade);
    websAspDefine("aspGetCharset", aspGetCharset);
    websFormDefine("WizardHandle", fromWizardHandle);
    websFormDefine("fast_setting_get", form_fast_setting_get);
    websFormDefine("fast_setting_pppoe_get", form_fast_setting_pppoe_get);
    websFormDefine("fast_setting_wifi_set", form_fast_setting_wifi_set);
    websFormDefine("fast_setting_pppoe_set", form_fast_setting_pppoe_set);
    websFormDefine("getWanConnectStatus", formGetWanConnectStatus);
    websFormDefine("getProduct", GetProduct);
    websFormDefine("fast_setting_internet_set", form_fast_setting_internet_set);
    websFormDefine("getSyncAccount", form_SyncAccount_get);
}
```

image-20220422113423088

上面这些都是通过 `goform` 来处理的，所以其访问形式为 <http://127.0.0.1:80/goform/TendaGetLongString> 这样的

哪个路径就交由哪个函数来处理。

下面分析可以由两方面展开：

分析各个功能点

简单来说就是将所有接口的代码过一遍，去分析参数从哪里来，有没有经过什么危险函数

这种的话效率比较低，我个人推荐第二种

通过危险函数来查找可利用点，利用逆向分析工具的交叉编译功能查找

这里放一张危险函数表

`dosystemcmd`

`system`

函数	严重性	解决方案
gets	最危险	使用 fgets (buf, size, stdin) 。这几乎总是一个  !
strcpy	很危险	改为使用 strncpy。
strcat	很危险	改为使用 strncat。
sprintf	很危险	改为使用 snprintf，或者使用精度说明符。
scanf	很危险	使用精度说明符， 或自己进行解析。
sscanf	很危险	使用精度说明符， 或自己进行解析。
fscanf	很危险	使用精度说明符， 或自己进行解析。
vfscanf	很危险	使用精度说明符， 或自己进行解析。
vsprintf	很危险	改为使用 vsnprintf，或者使用精度说明符。
vscanf	很危险	使用精度说明符， 或自己进行解析。
vsscanf	很危险	使用精度说明符， 或自己进行解析。
streadd	很危险	确保分配的目的地参数大小是源参数大小的四倍。
strecpy	很危险	确保分配的目的地参数大小是源参数大小的四倍。
strtrns	危险	手工检查来查看目的地大小是否至少与源字符串相等。
realpath	很危险（或稍小，取决于实现）	分配缓冲区大小为 MAXPATHLEN。同样，手工检查参数以确保输入参数不超过 MAXPATHLEN。
syslog	很危险（或稍小，取决于实现）	在将字符串输入传递给该函数之前，将所有字符串输入截成合理的大小。
getopt	很危险（或稍小，取决于实现）	在将字符串输入传递给该函数之前，将所有字符串输入截成合理的大小。
getopt_long	很危险（或稍小，取决于实现）	在将字符串输入传递给该函数之前，将所有字符串输入截成合理的大小。
getpass	很危险（或稍小，取决于实现）	在将字符串输入传递给该函数之前，将所有字符串输入截成合理的大小。
getchar	中等危险	如果在循环中使用该函数，确保检查缓冲区边界。
fgetc	中等危险	如果在循环中使用该函数，确保检查缓冲区边界。
getc	中等危险	如果在循环中使用该函数，确保检查缓冲区边界。
read	中等危险	如果在循环中使用该函数，确保检查缓冲区边界。
bcopy	低危险	确保缓冲区大小与它所说的一样大。
fgets	低危险	确保缓冲区大小与它所说的一样大。
memcpy	低危险	确保缓冲区大小与它所说的一样大。
snprintf	低危险	确保缓冲区大小与它所说的一样大。
strccpy	低危险	确保缓冲区大小与它所说的一样大。
strcadd	低危险	确保缓冲区大小与它所说的一样大。
strncpy	低危险	确保缓冲区大小与它所说的一样大。
vsnprintf	低危险	确保缓冲区大小与它所说的一样大。

根据上面的函数表来将危险函数过一下

下面是之前分析到的两个问题的思路，住这里不涉及exp， poc等脚本的编写， 还是以思路为主。

命令执行

过一遍dosystemcmd函数

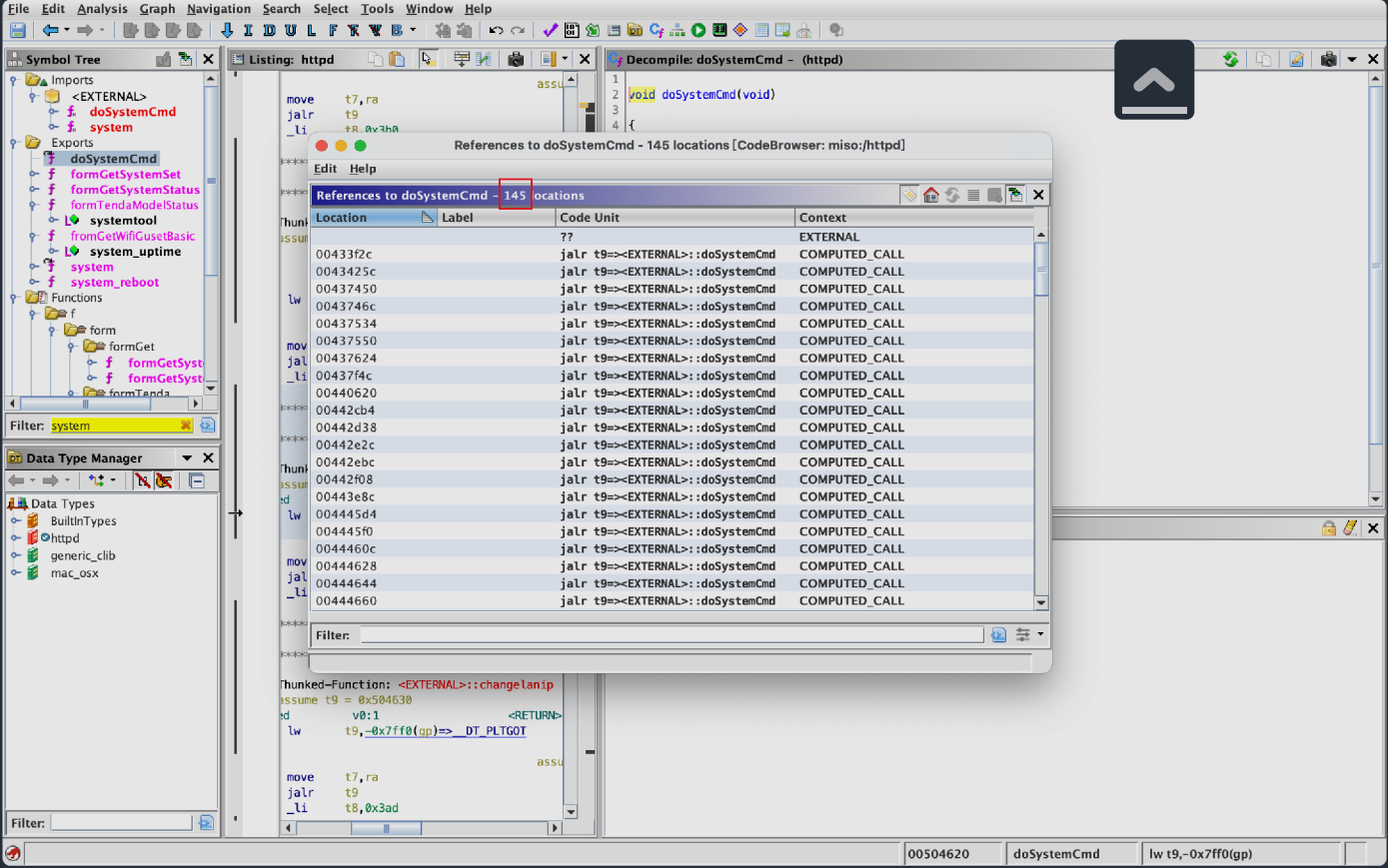


image-20220422141922087

可以看到有145次调用，感觉有漏洞的几率还是挺大的

注意点，尽量找form这类的函数，即上面说的自定义功能，有前后端交互

且危险函数的参数来自于前端参数



image-20220422142703774

`websGetVar` 就是从wp中获取其第二个参数对应的值，如果没有该参数，值默认为第三个参数。

上面可以看到这里pcVar1未作任何处理直接拼接到参数中。

这里就产生了命令执行，不多做赘述，各位有兴趣可自行复现。

溢出

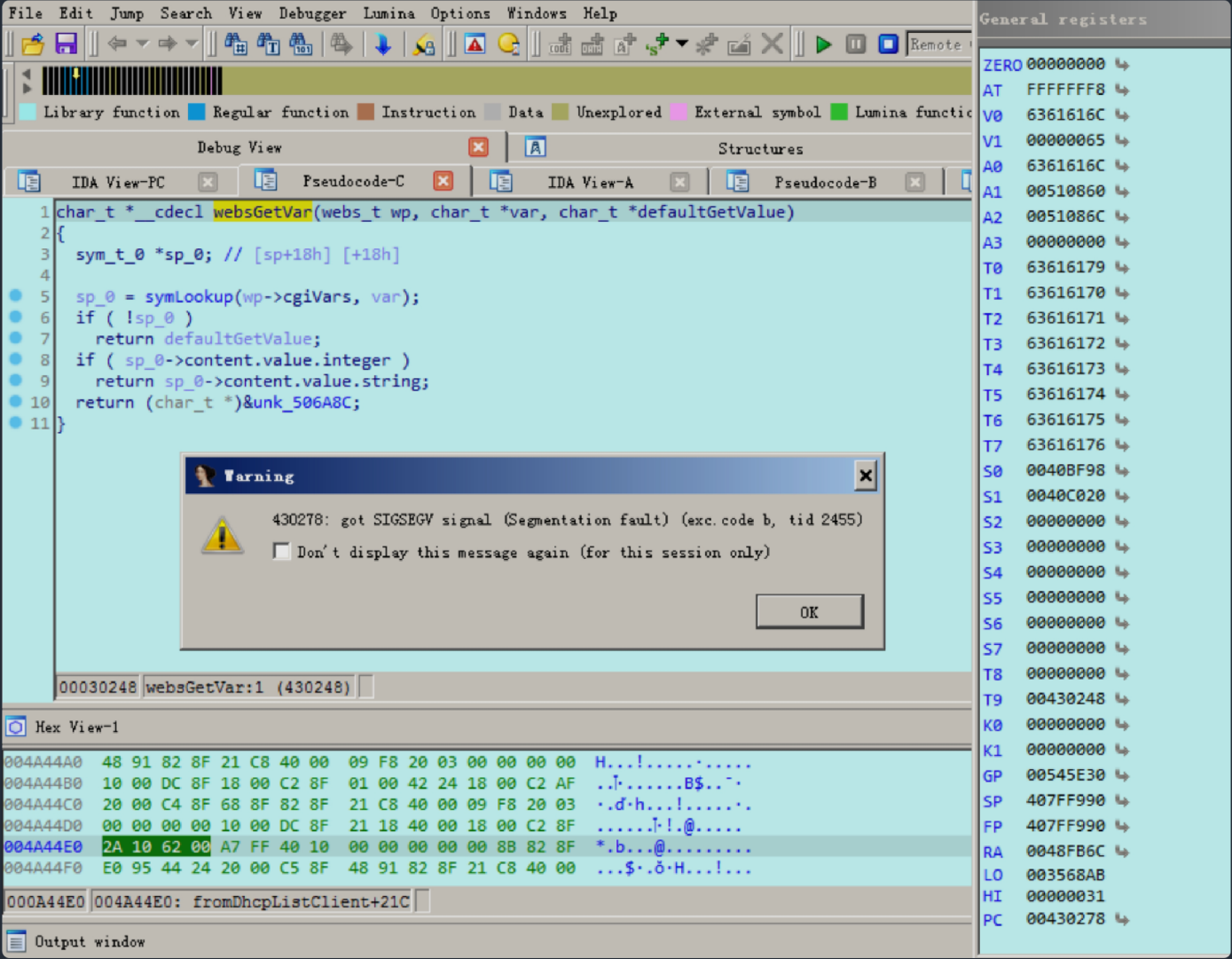
溢出是在strprintf函数的调用中发现的。

goform/NatStaticSetting路径访问到fromNatStaticSetting函数



img

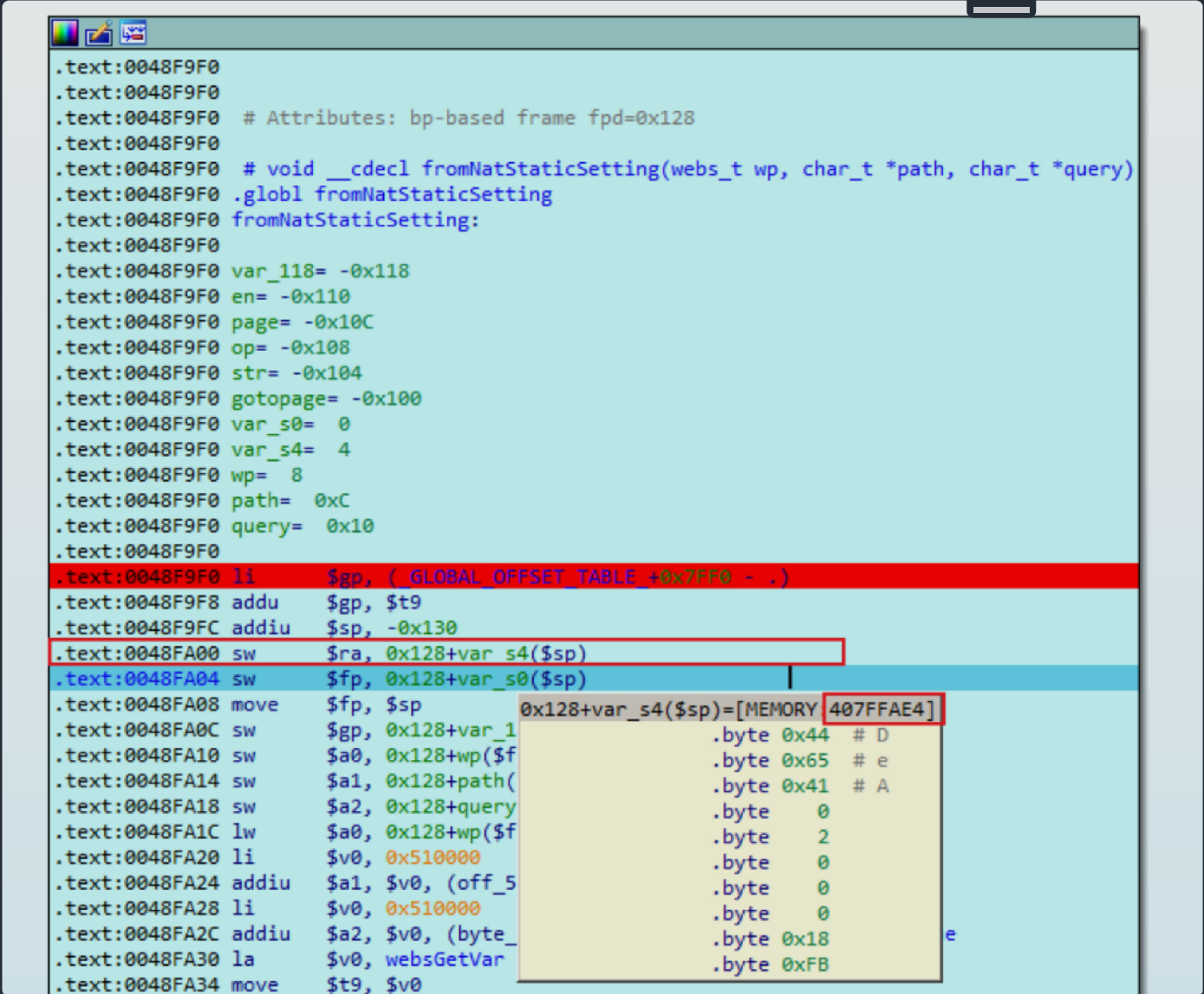
sprintf函数将page的参数给拼接字符串中，未做长度校验，导致溢出



img

复现过程：

断点到fromNatStaticSetting函数入口



img

将调用fromNatStaticSetting函数的返回地址放入0x407FFAE4 处

向下执行到第三个websGetVar函数获取page参数，然后向下执行sprintf函数，将page参数的内容拼接到gotopage内，由代码可知长度为256

参数初始化完毕后发现gotopage位置为0x407FF9E0

这里我们传入page参数为：

ABNF

ǎabpaabqaaabraabsaabtaabuaabvaabwaabxaabyaabzaacbaaccaacdaaceaacfaacgaachaaciaacjaacAAAA

执行完后

上面是一些思路之类的东西，第一个命令执行晚了几天。

[iot学习感悟](#)